Vedant Vartak

Homework 6: Document Storage and Multi-Layer Memory

Repository: https://github.com/vedantvartakwork/data260-3539

Report build commit: `fa4dd97dd912c31bcd51a47c30fe4f1104806f3d`

Resolved hw6 tagged implementation/evidence commit: `fa4dd97dd912c31bcd51a47c30fe4f1104806f3d`

Parameter	Value	Derivation / purpose
SID4	3539	last four student-ID digits
PORT_BASE	8839	8000 + (SID4 mod 900)
PREFIX	s3539	database/user namespace
SEED	3539	repeatable seed
VERIFY_SEED	263539	260000 + SID4
DOMAIN_ID	3	3539 mod 8
MongoDB	s3539_doc	document and memory database
Model	qwen3:8b via Ollama	local completion model
Embeddings	all-MiniLM-L6	local sentence-transformer
Hardware	MacBook Air M4, 10 cores, 16 GB	local execution

All application model calls were local. Raw JSONL/CSV, browser captures, verification, and the run log are committed under reports/hw06/.

1. Architecture and repository structure

Homework 6 extends the existing shared project rather than creating a separate app copy.

Layer	Implementation	Evidence
Document API	FastAPI routes + PyMongo repository	code/hw6/routes/documents.py; raw/database_crud_evidence.json
Conversation memory	messages, summaries, episodes	code/hw6/memory.py; raw/memory_api_evidence.json
Local generation	qwen3:8b through Ollama	raw/*_transcript.jsonl
Episodic retrieval	all-MiniLM-L6 + cosine top-k	episodes collection + memory response
Durable file memory	AGENT.md + MEMORY.md	MEMORY.original.md; file_memory_evidence.json
Verification	13 offline tests + artifact verifier	tests/test_hw06.py; verification.json

```
code/hw6/
  config.py, schemas.py, db.py, repositories.py, memory.py, file_memory.py
  routes/documents.py, routes/memory.py
reports/hw06/
  RUN_LOG.txt, METRICS.md, AI_USE.md, verification.json
  MEMORY.original.md, memory_probes.yaml, raw/, screenshots/
scripts/
  capture_hw06_*  run_hw06_memory_experiment.py  verify_hw06.py
```

2. Part 1 - MongoDB collection schema and validator

The database-level validator independently enforces required fields, enums, types, title length, and timestamps.

Relevant implementation

```
RECALL_VALIDATOR = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["title", "status", "category", "recalled_at", "created_at", "updated_at"],
        "properties": {
            "title": {"bsonType": "string", "minLength": 2, "maxLength": 100},
            "description": {"bsonType": ["string", "null"], "maxLength": 1000},
            "status": {"enum": ["active", "monitoring", "resolved"]},
            "category": {
                "enum": ["Produce", "Meat and Seafood", "Dairy and Refrigerated", "Packaged Foods"]
            },
            "recalled_at": {"bsonType": "date"},
            "created_at": {"bsonType": "date"},
            "updated_at": {"bsonType": "date"},
        },
    }
}
```

Verified output

```
database=s3539_doc
collections=episodes, messages, recall_documents, summaries
MongoDB validator bypass blocked=True code=121
seeded recall_documents=5
```

3. Part 1 - API models, CRUD, and invalid inputs

Pydantic rejects malformed requests before persistence; repository methods return correct status codes and JSON.

Relevant implementation

```
class RecallStatus(str, Enum):
    active = "active"
    monitoring = "monitoring"
    resolved = "resolved"


class RecallCategory(str, Enum):
    produce = "Produce"
    meat_seafood = "Meat and Seafood"
    dairy_refrigerated = "Dairy and Refrigerated"
    packaged_foods = "Packaged Foods"


class RecallDocumentBase(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)

    title: str = Field(min_length=2, max_length=100)
    description: str | None = Field(default=None, max_length=1000)
    status: RecallStatus = RecallStatus.active
    category: RecallCategory
    recalled_at: datetime

    @field_validator("description")
    @classmethod
    def blank_description_is_none(cls, value: str | None) -> str | None:
        return value or None


class RecallDocumentCreate(RecallDocumentBase):
    pass


class RecallDocumentUpdate(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)

    title: str | None = Field(default=None, min_length=2, max_length=100)
    description: str | None = Field(default=None, max_length=1000)
    status: RecallStatus | None = None
    category: RecallCategory | None = None
    recalled_at: datetime | None = None

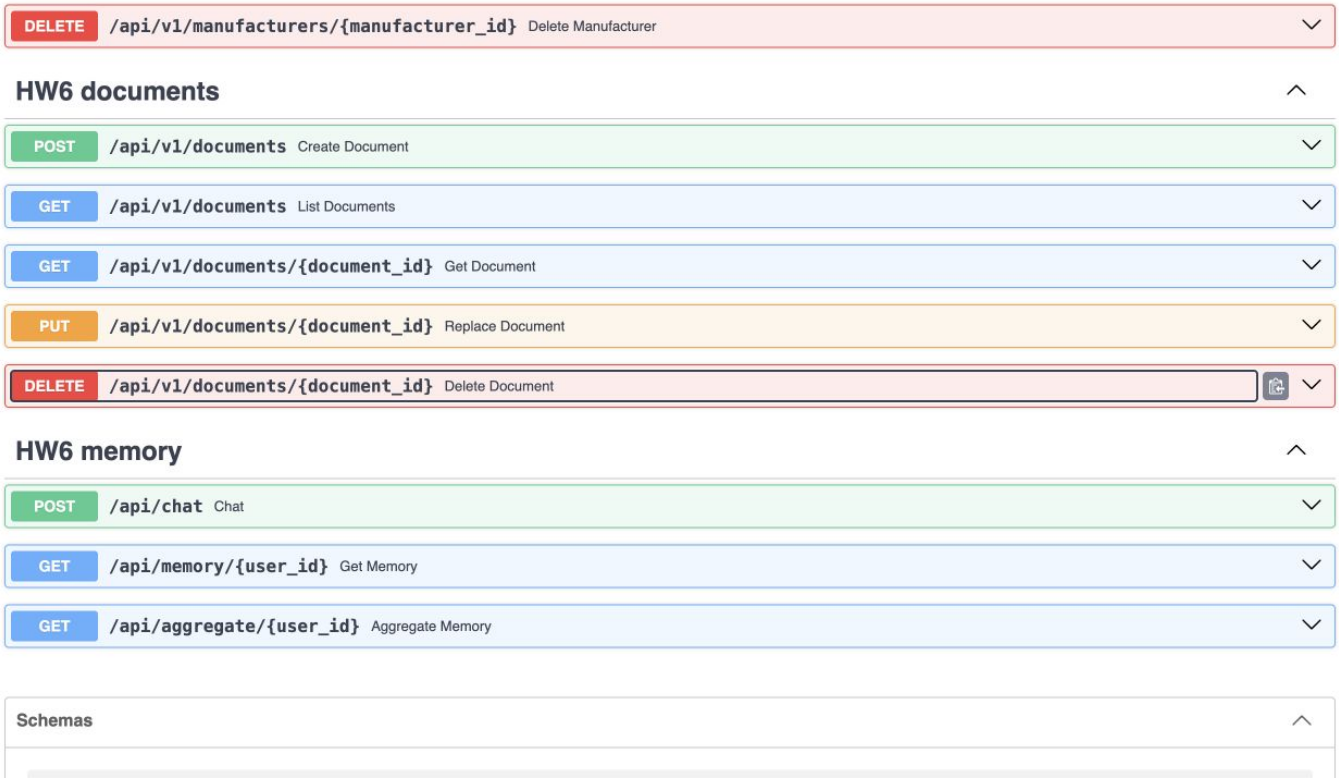
    @model_validator(mode="after")
    def require_one_field(self) -> "RecallDocumentUpdate":
        if not self.model_fields_set:
            raise ValueError("at least one field must be supplied")
        return self
```

Verified output

```
POST create -> 201
GET list -> 200
GET one -> 200
PUT update -> 200 status=monitoring
DELETE -> 204
GET after delete -> 404
missing required fields -> 422
invalid category enum -> 422
invalid datetime type -> 422
```

4. Part 1 - Live HW6 route map

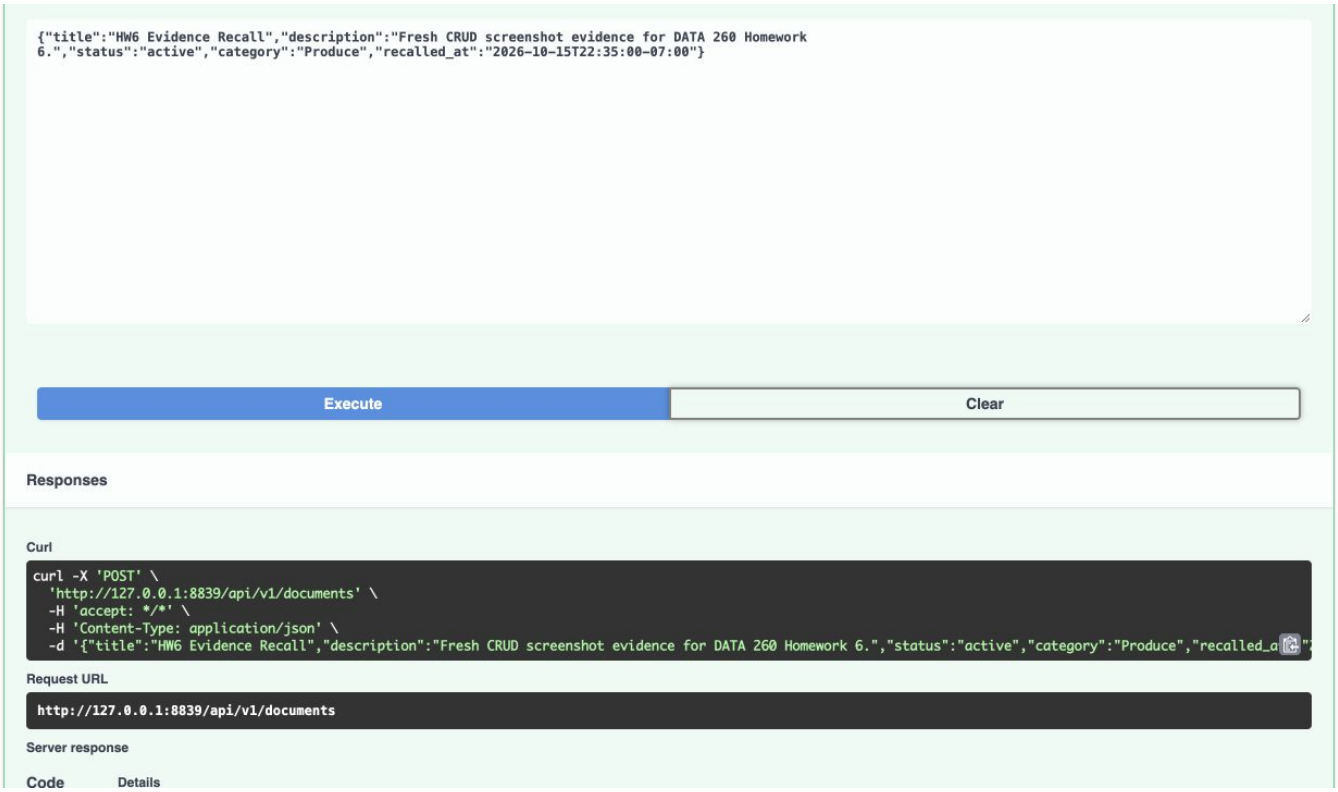
The running integrated API exposes POST, GET list, GET one, PUT, and DELETE document routes plus the three memory routes.



Genuine FastAPI Swagger UI capture from http://127.0.0.1:8839/docs; the required PUT route is visible.

5. Part 1 - POST document request

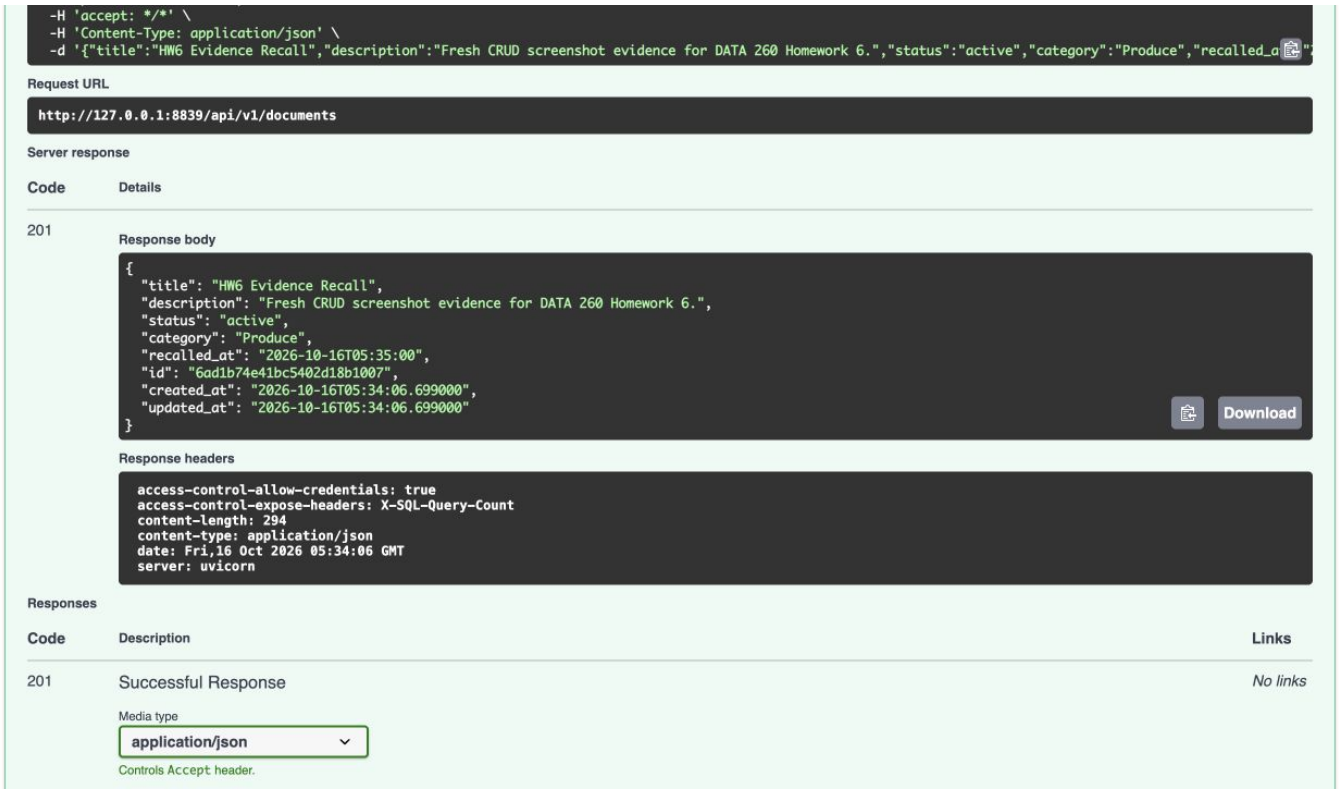
The live create request shows the exact validated JSON sent to the API.



POST /api/v1/documents with title, description, status, category, and recalled_at.

6. Part 1 - POST document response

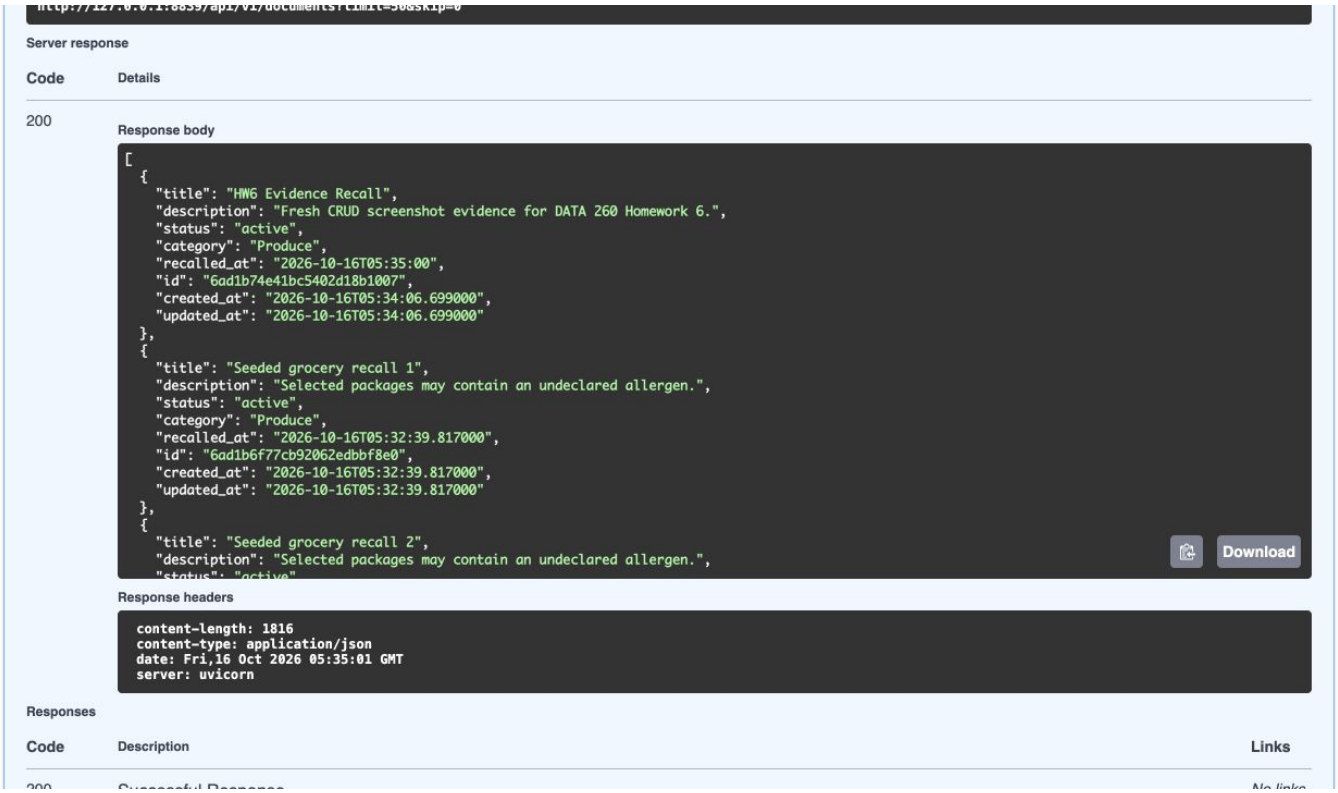
The same create call returned a MongoDB ObjectID, timestamps, and HTTP 201.



Successful POST response with the created document and status 201.

7. Part 1 - GET document list

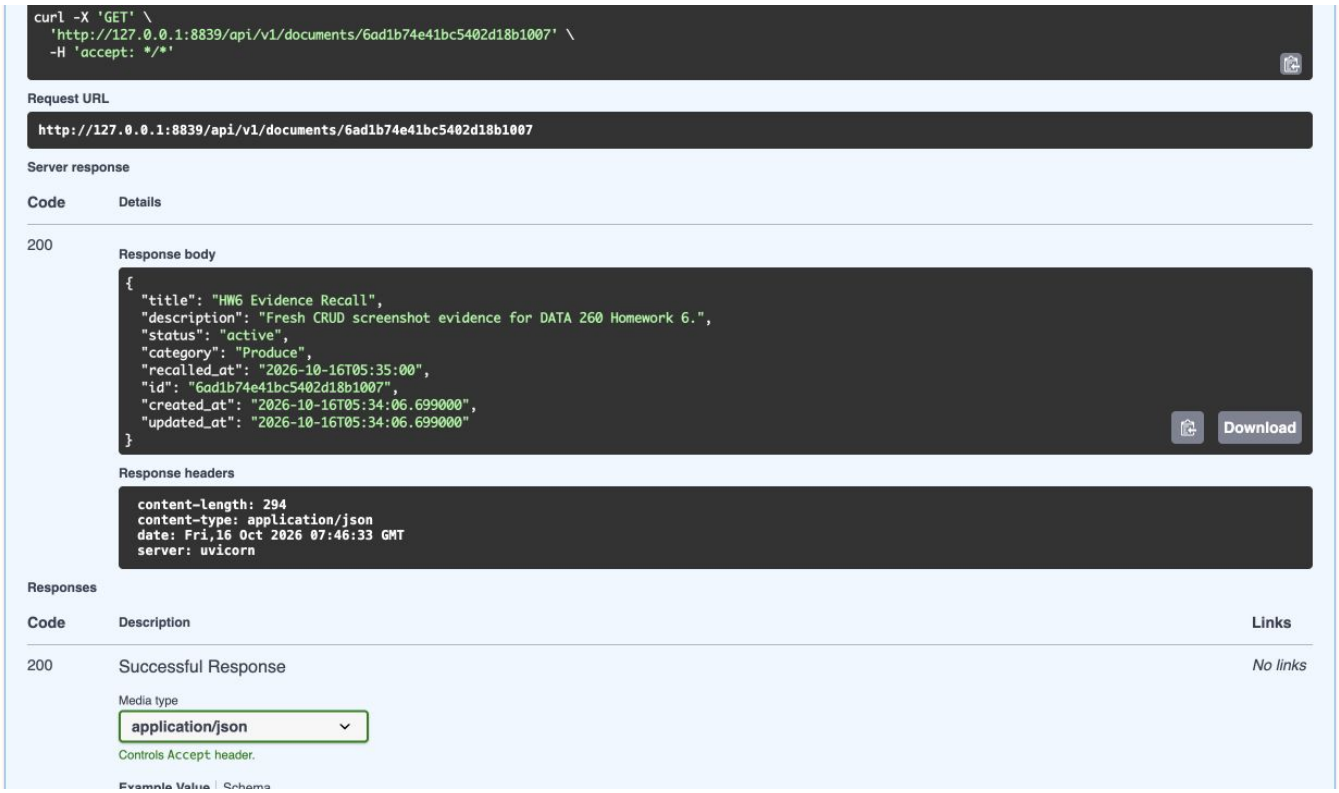
A real list call reached MongoDB and returned stored recall documents.



GET /api/v1/documents response with request URL and HTTP 200.

8. Part 1 - GET one document

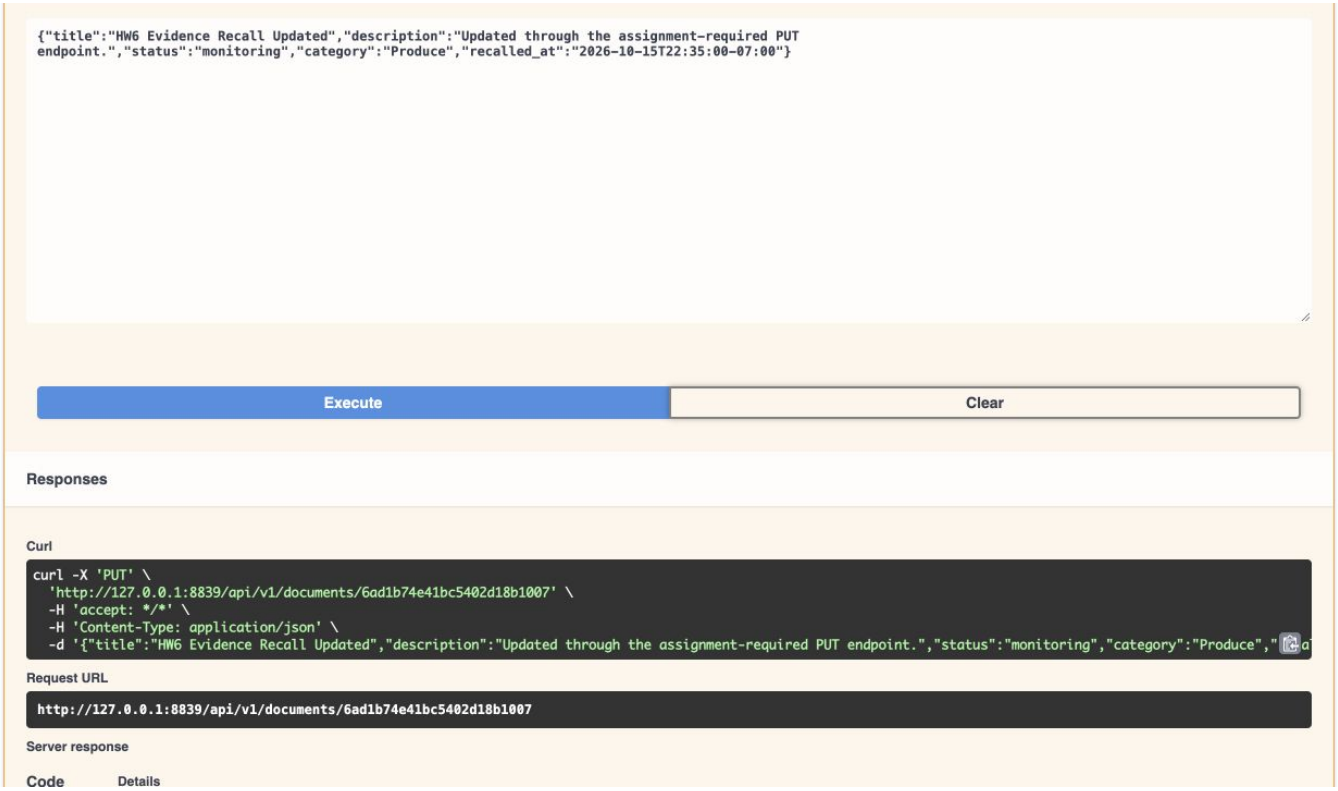
The exact created ObjectID is used in a successful read-by-id request.



GET /api/v1/documents/{document_id} with request URL, response body, and HTTP 200.

9. Part 1 - PUT document request

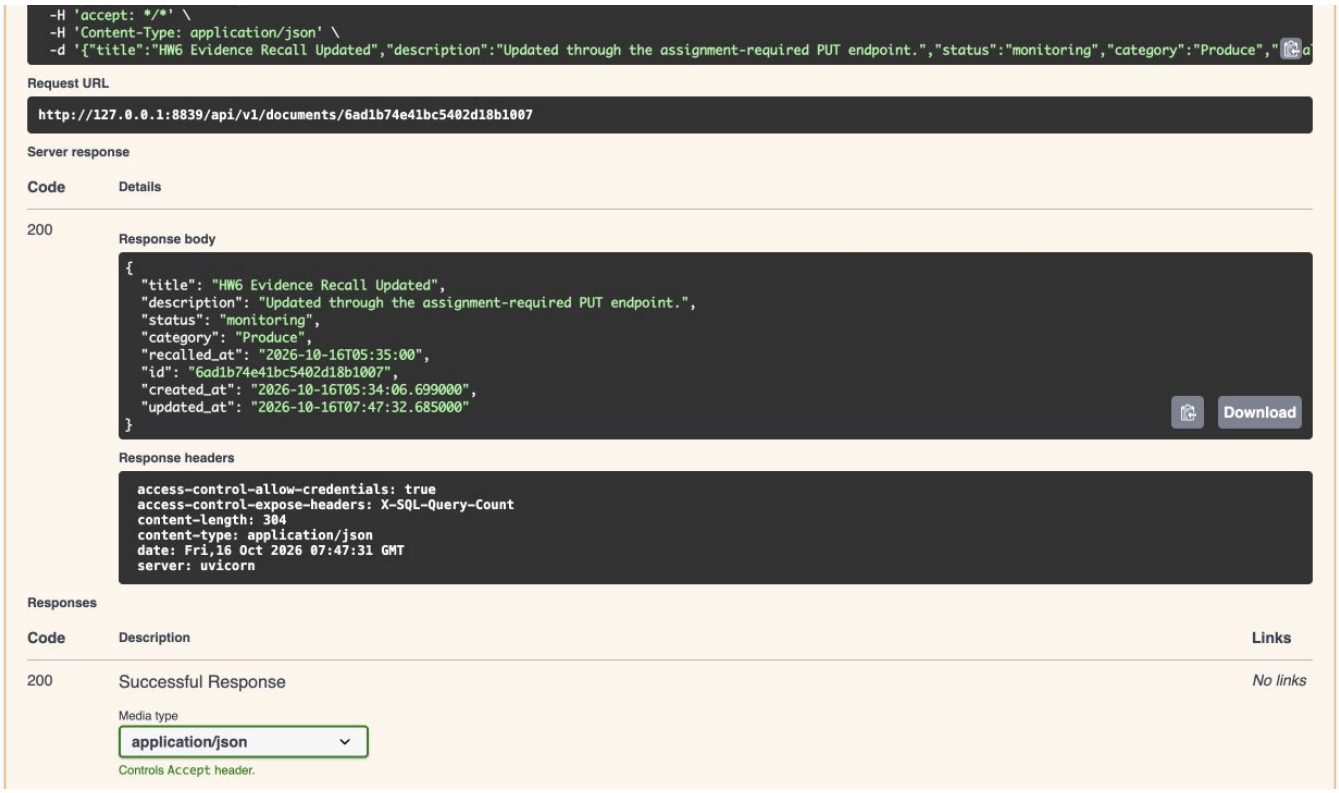
The required PUT update sends a validated status change for the same document.



PUT /api/v1/documents/{document_id} request body.

10. Part 1 - PUT document response

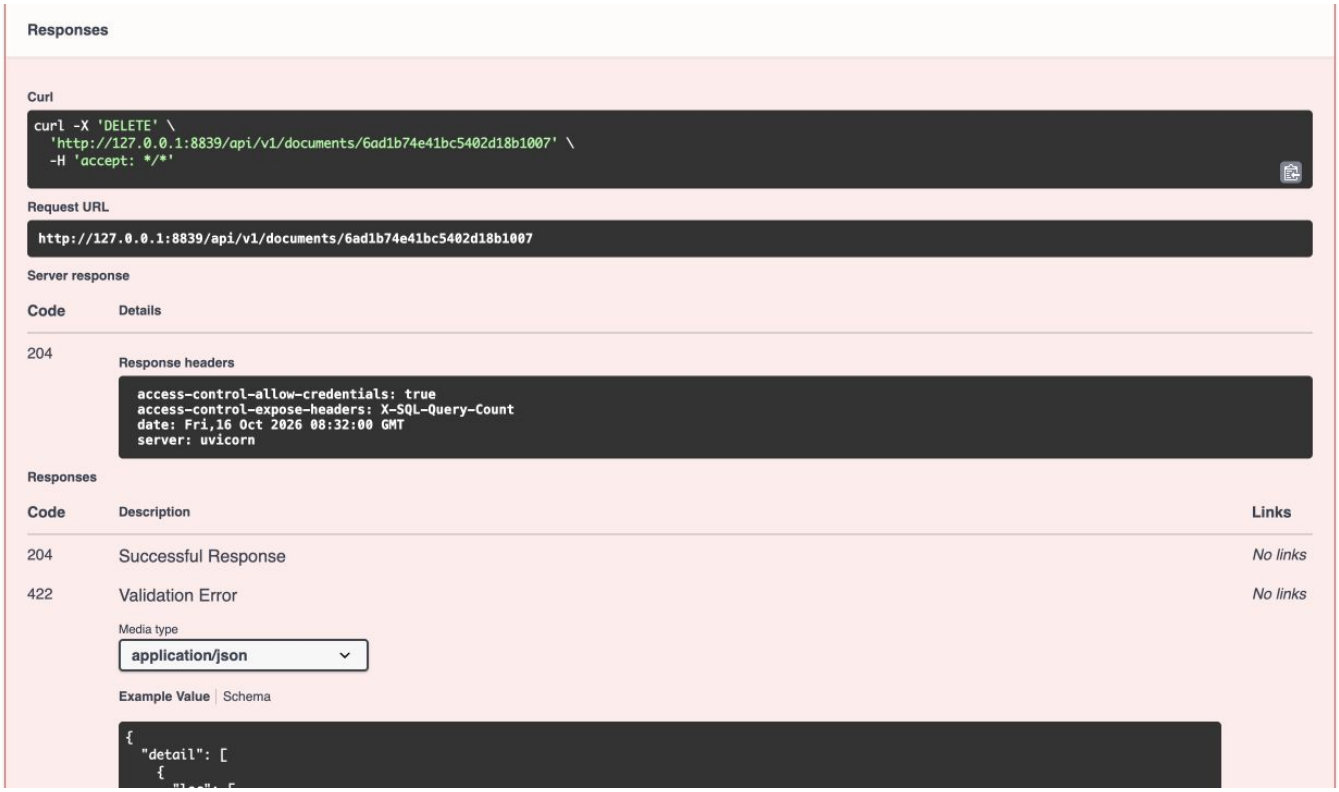
The required update returned the modified document and HTTP 200.



Successful PUT response for the same document ID.

11. Part 1 - DELETE document

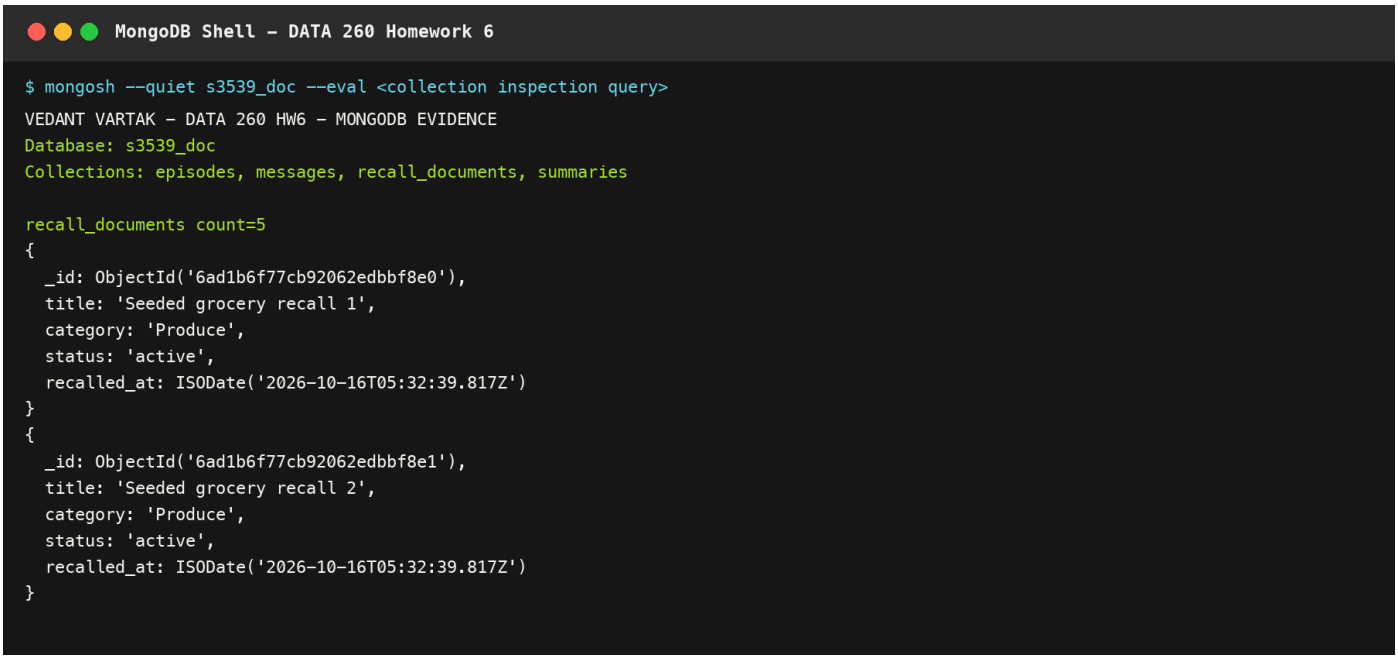
The same document was deleted through the live API.



DELETE /api/v1/documents/{document_id} with request URL and HTTP 204.

12. Part 1 - MongoDB recall collection

A live mongosh inspection confirms the configured database, collection, document count, and sample BSON documents.



Exact terminal output from s3539_doc.recall_documents; two sample documents are readable.

13. Part 2 - MongoDB memory collections

The same live mongosh inspection verifies messages, summaries, and episodes with counts and sample records.

MongoDB Shell – DATA 260 Homework 6

```
$ mongosh --quiet s3539_doc --eval <collection inspection query>
messages count=24
{
  user_id: 'vedant-hw6',
  session_id: 'demo-3539',
  role: 'user',
  content: 'My name is Vedant Vartak and my project prefix is s3539.'
}

summaries count=4
{
  user_id: 'vedant-hw6',
  scope: 'session'
}

episodes count=14
{
  user_id: 'vedant-hw6',
  fact: 'My name is Vedant Vartak and my project prefix is s3539.',
  importance: 0.65
}
```

Exact terminal output for the three required memory collections.

14. Part 2 - Multi-layer chat-memory pipeline

Each request saves the user message, loads bounded short-term memory, latest summaries and top-k episodes, calls local Ollama, saves the assistant reply, extracts up to three facts, and records telemetry.

Relevant implementation

```
    def chat(self, user_id: str, session_id: str, message: str) -> dict[str, Any]:
        if message.startswith("/") and self.file_memory is not None:
            command_reply = self.file_memory.command(
                message, self.repository.messages(user_id, session_id=session_id, limit=20)
            )
            if command_reply is not None:
                return {
                    "user_id": user_id,
                    "session_id": session_id,
                    "reply": command_reply,
                    "memory_used": {"short_term_count": 0, "short_term_messages": [], "session_summary": No
                    "usage": {"prompt_tokens": 0, "completion_tokens": 0, "latency_ms": 0.0, "model": "comm
                }

# ... bounded retrieval and local generation ...
        started = time.perf_counter()
        user_row = self.repository.add_message(user_id, session_id, "user", message)
        short_term = self.repository.messages(
            user_id,
            session_id=session_id,
            limit=self.short_term_n,
            before_id=user_row["id"],
        )
        session_row = self.repository.latest_summary(user_id, "session", session_id)
        lifetime_row = self.repository.latest_summary(user_id, "user")
        session_summary = session_row["text"] if session_row else None
        lifetime_summary = lifetime_row["text"] if lifetime_row else None
        episodes = self._retrieve_episodes(user_id, message)

# ... bounded retrieval and local generation ...
        system_prompt = self._system_prompt(session_summary, lifetime_summary, episodes)
        model_messages = [{"role": "system", "content": system_prompt}]
        model_messages.extend({"role": row["role"], "content": row["content"]} for row in short_term)
        model_messages.append({"role": "user", "content": message})

        completion = self.model.complete(model_messages, temperature=0.2)
        self.repository.add_message(user_id, session_id, "assistant", completion.content)

# ... bounded retrieval and local generation ...
        for fact, importance in extract_facts(message):
            self.repository.add_episode(user_id, session_id, fact, importance, self.embedder.embed(fact))

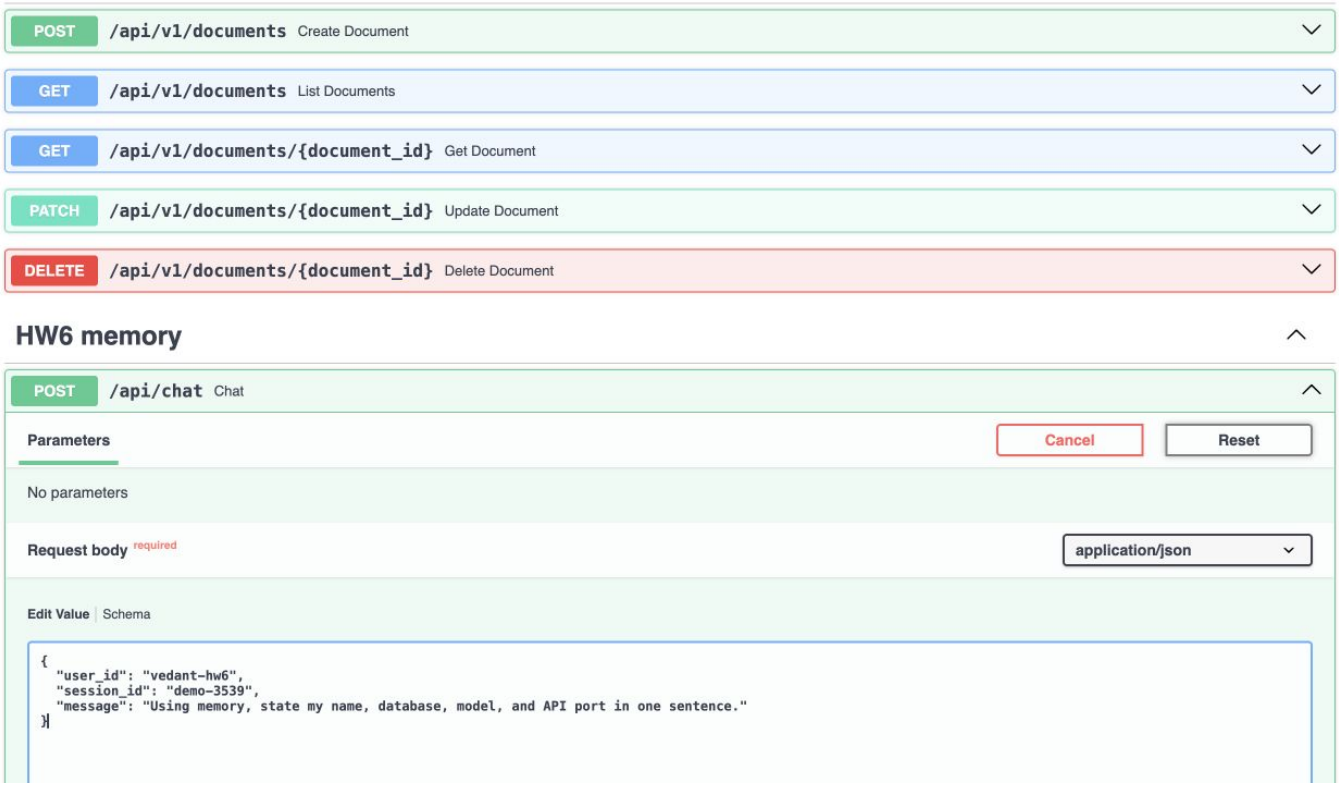
        user_count = self.repository.user_message_count(user_id, session_id)
        if user_count % self.summarize_every == 0:
            current = self.repository.messages(user_id, session_id=session_id, limit=30)
            self.repository.add_summary(user_id, session_id, "session", _summary_text(current))
        if user_count % (self.summarize_every * 3) == 0:
            recent = self.repository.recent_summaries(user_id, limit=10)
            if recent:
                text = " || ".join(row["text"] for row in reversed(recent))[-1200:]
                self.repository.add_summary(user_id, None, "user", text)
```

Verified output

```
POST /api/chat -> 200
short-term window: latest 12 prior messages
session summary: latest per-user/per-session summary
episodic memory: cosine-ranked top 3 facts
reply: Your name is Vedant Vartak, you are using the s3539_doc MongoDB
database, the qwen3:8b local model, and the API runs on port 8839.
```

15. Part 2 - Exact chat input

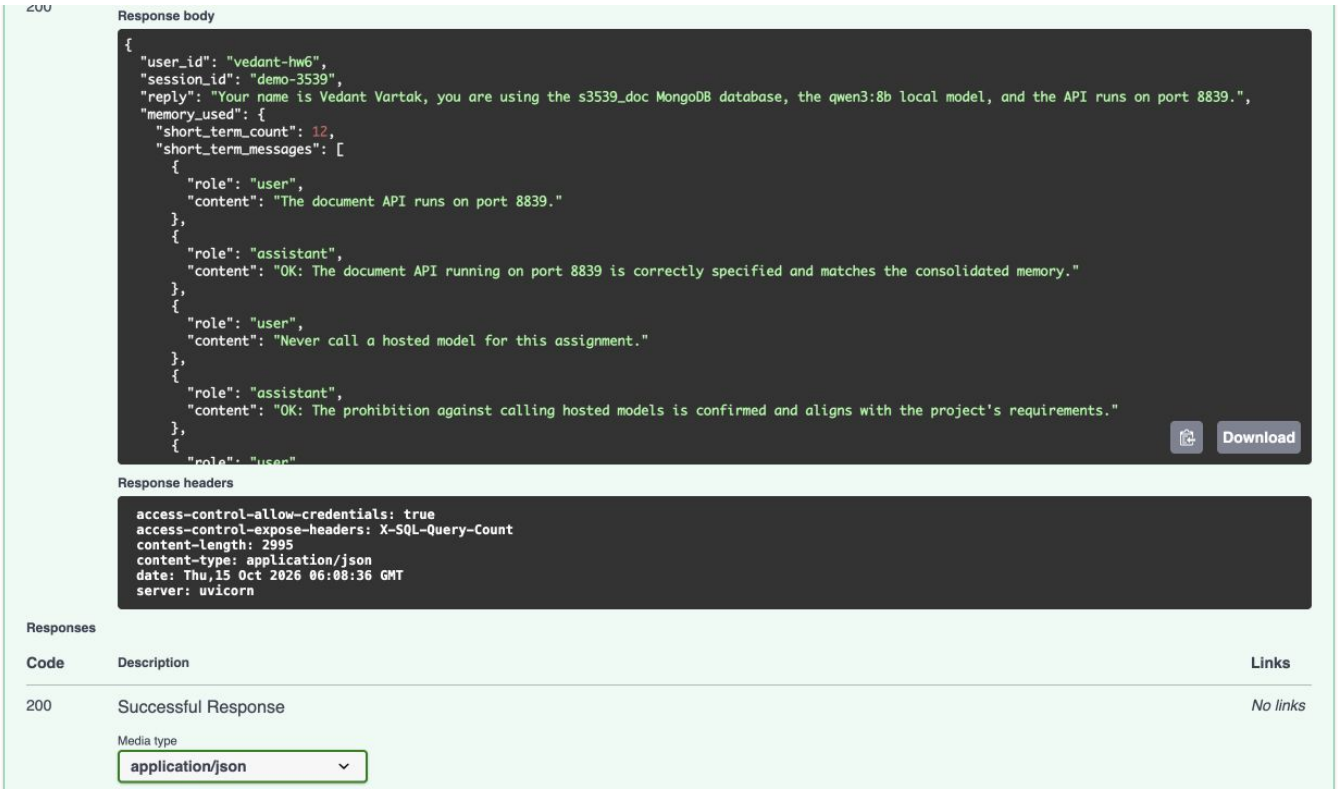
The user/session identifiers and memory probe are visible before execution.



Exact JSON submitted to POST /api/chat.

16. Part 2 - Memory-aware chat output

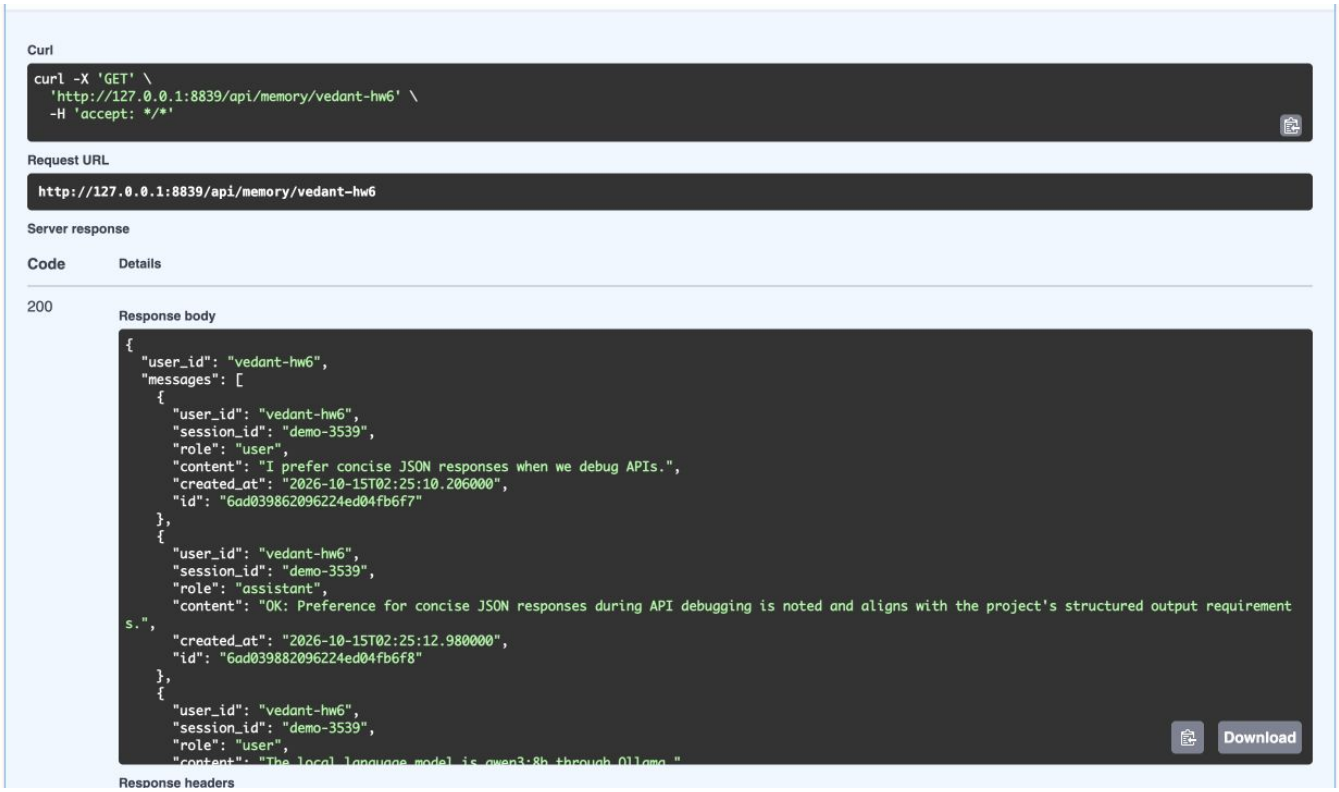
The output is from the same live call and includes a grounded answer plus the memory_used payload.



200 response: the local model recalled the name, database, model, and port.

17. Part 2 - GET memory

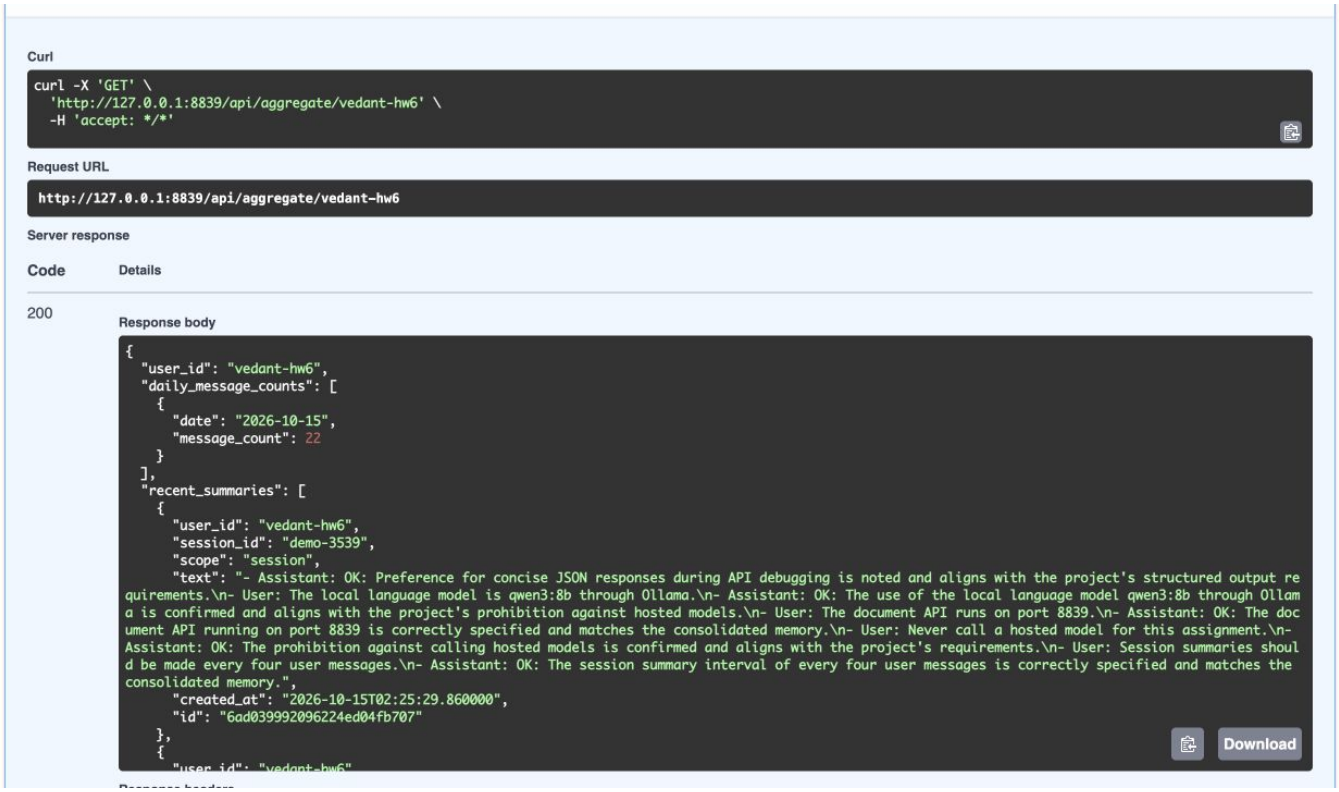
The memory endpoint exposes the latest 16 messages plus summaries and episodes for inspection.



Live request URL, status 200, and stored MongoDB messages.

18. Part 2 - GET aggregate

The aggregate endpoint returns daily message counts and recent summaries.



Live aggregate response: daily count and summary documents are visible.

19. Part 2 - Episodic facts, embeddings, and cosine retrieval

The production embedder is lazy and local; deterministic embeddings are used only by offline tests.

Relevant implementation

```
class SentenceTransformerEmbedder:
    """Lazy local embedding adapter; no network service is called at runtime."""

    def __init__(self, model_name: str = settings.embedding_model) -> None:
        self.model_name = model_name
        self._model: Any = None

    def embed(self, text: str) -> list[float]:
        if self._model is None:
            from sentence_transformers import SentenceTransformer

            self._model = SentenceTransformer(
                self.model_name,
                cache_folder=str(PROJECT_ROOT / ".cache" / "huggingface"),
            )
        vector = self._model.encode(text, normalize_embeddings=True)
        return [float(value) for value in vector]

def cosine_similarity(left: list[float], right: list[float]) -> float:
    if not left or len(left) != len(right):
        return 0.0
    denominator = math.sqrt(sum(v * v for v in left)) * math.sqrt(sum(v * v for v in right))
    return sum(a * b for a, b in zip(left, right)) / denominator if denominator else 0.0

def extract_facts(message: str) -> list[tuple[str, float]]:
    """Extract up to three short episodic facts from an incoming message."""
    sentences = [part.strip() for part in re.split(r"(?<=[.!?])\s+|\n+", message) if part.strip()]
    facts: list[tuple[str, float]] = []
    for sentence in sentences:
        if len(sentence) < 8:
            continue
        lower = sentence.lower()
        importance = 0.9 if any(word in lower for word in ("remember", "prefer", "deadline", "allergy", "re
        facts.append((sentence[:240], importance))
        if len(facts) == 3:
            break
    return facts
```

Verified output

```
Embedding model: all-MiniLM-L6 (local)
fact extraction cap: 3 facts/message
similarity: cosine(query_embedding, episode_embedding)
top-k: 3 (default)
example retrieved facts:
  0.681312  What are my name, database, local model, and API port?
  0.397206  Never call a hosted model for this assignment.
  0.363459  Remember that my grocery safety project uses MongoDB.
```

20. Part 3 - Controlled memory experiment

The same deterministic 30-message script and five probes ran in fresh sessions for N=4, N=12, N=30, and a compression/eviction policy.

Run	N	Summary event	Top-k	Mean prompt	Mean completion	Mean latency ms	Probe /5
window_n4	4	4	3	599.233	93.467	9302.267	3.8
window_n12	12	4	3	817.933	63.267	9386.976	3.8
window_n30	30	4	3	1248	63.133	14708.965	3.8
compact_policy	4	2	5	616.567	65.133	7731.462	4.6

All four runs contain 30 conversation turns plus five probe records. The raw per-turn JSONL and combined CSV preserve prompt/completion counts, system/long-term/short-term/episode components, latency, summaries, and eviction/compression behavior.

```
Probe rubric: 1 = absent/wrong, 3 = partially correct, 5 = exact and concise.
Five probes cover identity/config, durable preferences, an early critical fact,
recent context, and the explicit no-hosted-model constraint.
```

21. Part 3 - Compression, quality, and projected cost

The selected policy keeps four recent messages, summarizes every two user messages, and retrieves five episodes.

Compared with N=30, it reduced mean prompt tokens by **50.6%**, reduced mean latency from **14708.965 ms** to **7731.462 ms**, and improved mean probe score from **3.8** to **4.6**.

Policy	Projected cost for 10,000 conversations
window_n4	$10.67
window_n12	$11.34
window_n30	$15.64
compact_policy	$9.42

Best case is the compact policy because it preserves or improves quality with a much smaller prompt. Worst case is N=30 because it repeatedly resends the longest context. Pricing assumption: $1 per million input tokens and $5 per million output tokens.

22. Part 4 - AGENT.md categories and generated function

Project, stack, coding-convention, testing, and prohibited-action instructions are injected into the system prompt. The generated function is pure, typed, documented, and tested.

Relevant implementation

```
# Homework 6 Durable Agent Instructions

## Inherited HW1 code-review behavior

- Respond to code-review requests using bullet points only.
- Begin review bullets with `BUG:`, `RISK:`, `STYLE:`, or `OK:`.
- Cite a line number when the supplied code includes line numbers.
- Explain the concrete effect of every reported issue and do not invent problems.

## Project

- Reuse the existing DATA 260 repository and shared model adapter; do not create a second application copy.
- Keep SID-derived values deterministic: SID4 3539, PORT_BASE 8839, PREFIX s3539, and VERIFY_SEED 263539.

## Stack and architecture

- Use FastAPI, Pydantic v2, PyMongo, local MongoDB, and the existing local Ollama adapter.
- Keep configuration, persistence, schemas, routes, and orchestration in separate modules.

## Code conventions

- Add type hints and concise docstrings to public Python functions and classes.
- Emit structured JSON and keep stdout free of secrets or database credentials.

## Testing

- Provide deterministic offline tests that never require MongoDB, Ollama, or internet access.
- Run the HW6 verification script before tagging or packaging the submission.

## Prohibited actions

- Never call a hosted language model or put credentials in source control.
- Never delete or overwrite another homework's evidence, reports, or tags.
```

Verified output

```
normalize_memory_fact(' model ', ' qwen3:8b ')
-> - model: qwen3:8b

test_generated_normalizer_follows_agent_rules ... ok
```

23. Part 4 - /memory and /help commands

The command dispatcher exposes the required help text and returns the current durable memory with a verified meaningful-line count.

Relevant implementation

```
def command(self, command: str,
            recent_messages: list[dict[str, Any]]) -> str | None:
    name = command.strip().lower()
    if name == "/help":
        return (
            "Commands: /memory shows durable memory; "
            "/compact consolidates it; /dream reflects on "
            "recent messages; /help shows this list."
        )
    if name == "/memory":
        if not self.memory_path.exists():
            return "No durable MEMORY.md exists yet."
        text = self.memory_path.read_text(encoding="utf-8")
        count = len(self._meaningful_lines(text))
        return f"Durable memory has {count} meaningful lines.\n\n{text}"
```

Verified output

```
/help
Commands: /memory shows durable memory; /compact consolidates it;
/dream reflects on recent messages; /help shows this list.

/memory
Durable memory has 37 meaningful lines.

# Consolidated Homework 6 Memory
<!-- consolidated_at_utc: 2026-10-18T19:08:14.440808+00:00 -->
- student_name: Vedant Vartak
- sid4: 3539
- port_base: 8839
- prefix: s3539
- database: s3539_doc
...
```

24. Part 4 - /dream writes durable facts

The /dream command deterministically identifies useful long-term facts in recent user messages, rejects duplicates, and appends unique dream_fact entries to MEMORY.md.

Relevant implementation

```
    def _append_dream_facts(self, facts: list[str]) -> int:
        if not facts:
            return 0
        existing = self.memory_path.read_text(encoding="utf-8") if self.memory_path.exists() else "# Homewo
        existing_values = {
            line.split(":", 1)[1].strip().casefold()
            for line in self._meaningful_lines(existing)
            if line.startswith("- dream_fact_") and ":" in line
        }
        used_indexes = [
            int(match.group(1))
            for match in (re.match(r"- dream_fact_(\d+):", line) for line in existing.splitlines())
            if match
        ]
        next_index = max(used_indexes, default=0) + 1
        additions: list[str] = []
        for fact in facts:
            if fact.casefold() in existing_values:
                continue
            additions.append(normalize_memory_fact(f"dream_fact_{next_index}", fact))
            existing_values.add(fact.casefold())
            next_index += 1
        if additions:
            separator = "" if existing.endswith("\n") else "\n"
            self.memory_path.write_text(existing + separator + "\n".join(additions) + "\n", encoding="utf-8
        return len(additions)

        if name == "/dream":
            if not recent_messages:
                return "Dream reflection: there are no recent messages to reflect on."
            facts = self._dream_facts(recent_messages)
            added = self._append_dream_facts(facts)
            if not facts:
                return "Dream reflection: no durable long-term facts were identified in the recent messages
            return (
                f"Dream reflection: identified {len(facts)} useful long-term fact(s) "
                f"and added {added} new fact(s) to MEMORY.md."
            )
```

Verified output

```
dream=Dream reflection: identified 2 useful long-term fact(s) and added 2 new fact(s) to MEMORY.md.
dream_added_lines=['- dream_fact_1: my project database is MongoDB.', '- dream_fact_2: I prefer local qwen3
```

25. Part 4 - /compact consolidates durable memory

Consolidation triggers above 40 non-empty, non-comment memory lines or after 24 hours, keeps the newest conflicting value, and records every resolution.

Relevant implementation

```
def normalize_memory_fact(key: str, value: str) -> str:
    """Return one canonical, Markdown-safe memory fact.

    The function is intentionally pure and typed so consolidation can be tested
    without filesystem, network, model, or database access.
    """
    clean_key = " ".join(key.strip().split()).rstrip(":")
    clean_value = " ".join(value.strip().split())
    if not clean_key or not clean_value:
        raise ValueError("memory facts require both a key and a value")
    return f"- {clean_key}: {clean_value}"

    def needs_consolidation(self, now: datetime | None = None) -> bool:
        if not self.memory_path.exists():
            return False
        now = now or datetime.now(timezone.utc)
        count = len(self._meaningful_lines(self.memory_path.read_text(encoding="utf-8")))
        age_seconds = now.timestamp() - self.memory_path.stat().st_mtime
        return count > 40 or age_seconds > 24 * 60 * 60

# Inside consolidate():
        for key, occurrences in values.items():
            unique_options = list(dict.fromkeys(occurrences))
            chosen = occurrences[-1]
            output.append(normalize_memory_fact(key, chosen))
            if len(unique_options) > 1:
                conflicts += 1
                rejected = "; ".join(option for option in unique_options if option != chosen)
                output.append(f"- Resolution for {key}: kept '{chosen}' as the newest value; rejected '{rej

        if name == "/compact":
            result = self.consolidate()
            return (
                f"Compacted MEMORY.md from {result['before']} to {result['after']} meaningful lines; "
                f"removed {result['duplicates_removed']} duplicate lines and documented {result['conflicts_
            )
```

Verified output

```
needs_consolidation_before=True
meaningful_lines_before=55
meaningful_lines_after=37
compact=Compacted MEMORY.md from 55 to 37 meaningful lines; removed 18 duplicate lines and documented 3 cor
incorrect_resolutions=[]
```

26. Part 4 - Original vs. consolidated durable memory

The committed original has 55 meaningful lines, at least three duplicate groups, and three contradictions. The consolidated file has 37 meaningful lines and 39 physical lines, both below 40.

Original excerpt

```
# Original 55-line Homework 6 Memory Fixture
- student_name: Vedant Vartak
- sid4: 3539
- port_base: 8839
- prefix: s3539
- database: s3539_doc
- api_framework: FastAPI
- database_driver: PyMongo
- local_model: qwen3:8b
- verify_seed: 263539
- domain_id: 3
- preferred_output: JSON
...
- report_name: Vartak_HW6.pdf
- tag_name: hw6
- local_model: qwen3:8b
- deployment: local only
- preferred_output: JSON
- short_term_window: 12
- summary_interval: 4
- episodic_top_k: 3
```

Consolidated excerpt

```
# Consolidated Homework 6 Memory
<!-- consolidated_at_utc: 2026-10-18T19:08:14.440808+00:00 -->
- student_name: Vedant Vartak
- sid4: 3539
- port_base: 8839
- prefix: s3539
- database: s3539_doc
- api_framework: FastAPI
- database_driver: PyMongo
- local_model: qwen3:8b
- verify_seed: 263539
- domain_id: 3
- preferred_output: JSON
- test_mode: offline deterministic
- short_term_window: 12
- Resolution for short_term_window: kept '12' as the newest value; rejected '8' as conflicting or outdated.
- summary_interval: 4
- Resolution for summary_interval: kept '4' as the newest value; rejected '6' as conflicting or outdated.
```

Audited resolutions: short_term_window=12 (rejected 8), summary_interval=4 (rejected 6), episodic_top_k=3 (rejected 5). No incorrect resolution was found.

27. Part 4 - MongoDB memory versus file memory

Criterion	MongoDB	AGENT.md / MEMORY.md
Inspectability	Queryable collections with timestamps, embeddings and indexes	Human-readable, diffable and reviewable in Git
Restart persistence	Persists in the MongoDB volume	Persists with the repository
Scalability	Many users/sessions and aggregation	Small set of durable project facts
Best use	Conversation history, summaries, episodic retrieval	Must-not-forget rules and confirmed configuration

The hybrid prompt is intentional: MongoDB provides scalable retrieval, while the files provide compact human-auditable constraints.

28. Verification - 13/13 offline tests and artifact checks

Tests avoid external APIs and use deterministic repositories/models. Live MongoDB and Ollama evidence is verified separately.

```
test_chat_request_rejects_unsafe_identifier ... ok
test_compaction_merges_duplicates_and_documents_conflict ... ok
test_cosine_and_fact_extraction ... ok
test_document_schema_rejects_missing_enum_and_type ... ok
test_document_schema_valid_and_default ... ok
test_document_update_exposes_required_put_route ... ok
test_document_update_requires_a_field ... ok
test_episodic_fact_is_stored_and_retrieved ... ok
test_file_commands ... ok
test_generated_normalizer_follows_agent_rules ... ok
test_memory_aggregate_counts_messages ... ok
test_periodic_session_summary_is_created ... ok
test_short_term_window_is_bounded ... ok

13/13 tests passed.
```

Verifier check	Pass	Details
offline_tests	True	13/13 named deterministic tests passed
python_compile	True	compileall passed
required_files	True	all required source, report, memory, probe, and disclosure files present
mongodb_crud_evidence	True	POST/list/read/PUT/DELETE, 422 validation, and MongoDB validator verified
memory_experiment	True	4 runs x (30 conversation turns + 5 probes)
live_memory_evidence	True	20 messages, 2+ summaries, and 10+ episodic facts verified
file_memory_consolidation	True	55 meaningful lines consolidated to 37; final MEMORY.md has 39 physical lines

29. AI-use disclosure

1. I used an AI assistant to help interpret the assignment, implement and debug code, write tests, and organize the report evidence.

2. I rejected suggestions that did not match the stated FastAPI, MongoDB, local-model, or evidence requirements.

3. I verified the work with deterministic tests, raw experiment records, API responses, and manual review of the required files.

4. I kept responsibility for the submitted implementation, evidence, explanations, and final verification.

I verified the implementation with named tests, MongoDB validation, raw 30-turn experiment records, real local-model responses, and exact request/response captures. The Homework 6 application did not call a hosted model.